

Peridot: Artifact Guide

Middleware 2026 Artifact Evaluation

2026-09-27

Contents

1	What this artifact contains	2
2	Badges claimed	2
2.1	Availability	2
3	System requirements	2
4	Getting Started Guide	2
4.1	1. Build the image and start the object store	3
4.2	2. Confirm the runtime and a context chain work	3
4.3	3. Confirm the analysis path works	3
5	Step-by-Step Instructions	3
5.1	Running everything at once	3
5.2	Scale	4
5.3	Figure 3: interposition overhead (§3.2)	4
5.4	Figure 5: dynamic I/O provisioning (§4.1)	4
5.5	Table 1: asynchronous versus synchronous offload (§4.2)	4
5.6	Figure 6: in-runtime Parquet batching (§4.2)	4
5.7	Figure 7: WASI hostcall batching (§4.3)	5
6	Interpreting the output	5
7	What this artifact does and does not reproduce	5
7.1	Reproduced	5
7.2	Not reproduced, and why	6
7.3	Magnitudes	6
8	Deviations from the paper’s method	6
9	Layout	6
9.1	Guest modules	6
10	Running without Docker	7
11	The containerd shim	7
12	Acknowledgements	7

Artifact for Middleware ’26. The reference manual for Peridot itself is [docs/peridot.md](https://docs.peridot.md).

Peridot runs a WebAssembly module under WASI preview 1 and replaces the implementation of individual hostcalls with a **context**: a link in a chain between the guest and the real WASI implementation. Contexts are selected at run time in a configuration file; the guest is not recompiled.

1 What this artifact contains

Source	The runtime, the Overseer, every context, the containerd shim, and the ImageNet workload.
Build	<code>artifact/Dockerfile</code> and <code>artifact/docker-compose.yml</code> , which also starts an S3-compatible object store.
Guests	Precompiled <code>wasm32-wasip1</code> modules in <code>artifact/guests/</code> ; <code>artifact/bin/build-guests.sh</code> rebuilds them.
Experiments	One script per figure in <code>artifact/experiments/</code> , each writing a CSV to <code>artifact/results/</code> .
Analysis	One matplotlib script per figure in <code>artifact/plots/</code> , writing PDF and PNG to <code>artifact/figures/</code> .
Sample data	CSVs in <code>artifact/results/reference/</code> , recorded on the packaging machine, so the figures can be regenerated without running anything. Not the paper's measurements; see its README .
Claims	CLAIMS.txt : each claim, whether this artifact supports it, and the command that checks it.

2 Badges claimed

Artifacts Available, Artifacts Functional, and Results Reproduced.

Results Reproduced covers Figures 3, 5, 6 and 7, and the two rows of Table 1 that need neither a native IBM GEDS installation nor a FUSE mount. Figure 4 is not reproduced. See [What this artifact does and does not reproduce](#).

2.1 Availability

Source repository: <https://github.com/pablogs98/Peridot>

Licence: [Apache License 2.0](#). Third-party material that this licence does not cover is listed in [THIRD-PARTY-NOTICES.md](#).

Before submission, fill in the line below and delete this note. The Artifacts Available badge asks for a public, archival location. A GitHub URL alone is not archival, because history can be rewritten or the repository removed; mint a Zenodo DOI from the tagged release and cite it here.

- Archived release (DOI): `TODO`

3 System requirements

Docker and Docker Compose. Four cores, 8 GiB of RAM and about 10 GB of free disk. On Docker Desktop the disk counts against the VM's limit, not the host's. For timings comparable to the paper, use an x86-64 Linux host.

The paper's numbers were measured on AWS EC2 running Ubuntu 22.04: an `m7i.2xlarge` (8 vCPU, 32 GiB) for §4.1, and a `t3.xlarge` (4 vCPU, 16 GiB, 5 Gbps) with Amazon S3 for §4.2 and §4.3. This artifact uses a local MinIO container instead, at reduced scale.

4 Getting Started Guide

About 10 minutes, most of it download. Confirms the artifact builds, runs and produces figures. No experiment is run to completion; that is [Step-by-Step Instructions](#).

4.1 1. Build the image and start the object store

```
cd <repository root>
docker compose -f artifact/docker-compose.yml up -d
```

Builds the runtime, the Overseer, every context, the plugin and the external batcher from source, and starts MinIO with the bucket created.

4.2 2. Confirm the runtime and a context chain work

```
docker compose -f artifact/docker-compose.yml run --rm peridot experiments/smoke.sh
```

Expected output ends with:

```
[INFO peridot] Context chain: ["counter"]
[INFO peridot_counter_ctx] Total bytes written: 67108982
==> PASS - counter intercepted the write path, 67108982 bytes counted
```

The last digits vary: the total is the guest's 64 MiB payload plus its own progress lines.

4.3 3. Confirm the analysis path works

```
docker compose -f artifact/docker-compose.yml run --rm peridot \
python3 plots/plot_all.py --reference
```

Regenerates all four figures from the sample CSVs in `results/reference/`, writing PDF and PNG into `artifact/figures/` on the host. They correspond to Figures 3, 5, 6 and 7 of the paper, and each script prints the relevant ratios to stdout.

5 Step-by-Step Instructions

Run these from `/artifact` inside the container:

```
docker compose -f artifact/docker-compose.yml exec peridot bash
```

Claim	Script (in <code>experiments/</code>)	Runtime
§3.2, Fig. 3: interposition costs essentially nothing	<code>exp_overhead.sh</code>	~2 min
§4.1, Fig. 5: bandwidth tracks declared demand	<code>exp_io.sh</code>	~6 min
§4.2, Tab. 1: async offload beats sync offload	<code>exp_storage.sh</code>	~2 min
§4.2, Fig. 6: in-runtime batching beats an external batcher	<code>exp_storage.sh</code>	~8 min
§4.3, Fig. 7: hostcall batching raises write IOPS	<code>exp_hostcall_batching.sh</code>	~2 min

[CLAIMS.txt](#) restates each claim in the paper's own words, including the functional ones no single command covers.

5.1 Running everything at once

```
docker compose -f artifact/docker-compose.yml run --rm peridot \
experiments/run_all.sh
```

About 15 minutes at reduced scale. Runs every experiment, regenerates every figure, and prints a summary of what ran and what was skipped. CSVs land in `artifact/results/` and figures in `artifact/figures/` on the host.

5.2 Scale

Experiments run a reduced configuration by default. `PERIDOT_AE_FULL=1` switches to the paper’s parameters, which need the paper’s hardware and run for hours:

```
docker compose -f artifact/docker-compose.yml run --rm \
  -e PERIDOT_AE_FULL=1 peridot experiments/run_all.sh
```

The reduced configuration lowers volumes, iteration counts and repetitions. Each script states both parameters at the top.

5.3 Figure 3: interposition overhead (§3.2)

```
experiments/exp_overhead.sh          # ~2 min
python3 plots/plot_overhead.py
```

Times `fd_write`, `fd_pwrite`, `fd_read` and `fd_pread` at 1 KiB inside the guest, with an empty context chain and with `syscall-batching` at `num_writes: 1`, which disables batching and leaves only interposition.

Expected. Bars of equal height within the error bars; the printed percentages straddle zero. Observed -0.8% to +6.9%.

5.4 Figure 5: dynamic I/O provisioning (§4.1)

```
experiments/exp_io.sh                # ~6 min
python3 plots/plot_io.py
```

Four colocated modules write concurrently under a global 1 Gbps policy with demands of 100/200/300/400 Mbps, once without the Overseer (each pinned to an equal quarter) and once with it.

Expected. Upper subplot: the four lines overlap regardless of demand. Lower subplot: they separate in proportion to it. Observed 224/225/229/227 Mbps against 74/174/280/378 Mbps.

At the paper’s demands the total (100+200+300+400) equals the policy exactly, so max-min fair share grants each module its demand and a module that finishes frees capacity no other module is asking for. The allocation is therefore flat and demand-proportional rather than stepping up over time. Oversubscribing the policy shows the reallocation:

```
IO_DEMANDS_MBPS="100 900 900 900" experiments/exp_io.sh
```

Each module is then clamped below its demand, and the clamps loosen as modules finish. The policy is unit-tested in `crates/peridot-overseer/src/policy.rs` (`cargo test -p peridot-overseer`).

5.5 Table 1: asynchronous versus synchronous offload (§4.2)

```
experiments/exp_storage.sh          # ~2 min for this part
python3 plots/plot_batch.py
```

Runs the ImageNet guest, which only opens, writes and closes ordinary files, with the `s3` context in asynchronous mode (uploads spawned) and in synchronous mode (`sync: true`, each upload awaited inside `fd_write`).

Expected. `s3-async-ctx` ahead of `s3-sync-ctx`. Observed 107 vs 49 MB/s, a factor of 2.2 against the paper’s 5.1x on Amazon S3; `artifact/env.example` shows how to point at a remote store. The `s3fs` and `geds-ctx` rows are not run, see [Not reproduced](#).

5.6 Figure 6: in-runtime Parquet batching (§4.2)

```
experiments/exp_storage.sh          # ~8 min for this part
python3 plots/plot_batch.py
```

The same guest at batch sizes 2 to 128. The Peridot arm uses the `batch` context, which groups files and uploads each group as one Parquet object from inside the runtime; the Wasmtime arm runs with no context

and a separate `tensor_batcher` process. Both arms are timed until every batch is durably in the object store, since the external batcher keeps working after the guest exits.

Expected. Peridot ahead at every batch size, largest margin at the smallest. Observed 2.05x at batch 2 and 1.37x at batch 128, against the paper’s 1.15x at the smallest batch.

5.7 Figure 7: WASI hostcall batching (§4.3)

```
experiments/exp_hostcall_batching.sh # ~2 min
python3 plots/plot_hostcall_batching.py
```

Sequential writes of 64 B, 256 B, 1 KiB and 4 KiB while the `syscall-batching` context buffers `num_writes` of them and flushes each group as one host write, sweeping `num_writes` from 1 (batching off) to 1024.

Expected. IOPS rise with batch size, most at 64 B, least at 4 KiB, flattening past 128. The multiplier scales with the cost of a host `write`, so it is far larger inside a VM than the paper’s 4.90x on EC2.

6 Interpreting the output

Every experiment produces three things.

1. **A CSV in `results/`**, one row per measurement; aggregation happens in the plot script.

<code>overhead.csv</code>	<code>arm,rep,op,bytes,iterations,</code> <code>mean_us,p50_us,p99_us,stddev_us</code>
<code>io.csv</code>	<code>arm,module,demand_mbps,t_seconds,written_bytes</code>
<code>storage_table1.csv</code>	<code>setup,rep,images,seconds,throughput_mbs</code>
<code>batch.csv</code>	<code>arm,batch_size,rep,images,seconds,throughput_mbs</code>
<code>hostcall_batching.csv</code>	<code>io_size,batch_size,writes_per_run,runs,</code> <code>mean_kiops,stddev_kiops</code>

`io.csv` records cumulative bytes; the plot script differentiates them into a bandwidth series. The `arm` or `setup` column separates the baseline from Peridot and is the comparison each figure makes.

2. **A figure in `figures/`**, as both PDF and PNG.

3. **The numbers the claim rests on, printed to `stdout`**. For example `plots/plot_hostcall_batching.py` prints:

```
speedup over no batching (num_writes = 1):
    64 B: baseline    16.0 K -> best    8236.5 K at num_writes=1024 (513.53x)
```

and `plots/plot_batch.py` prints Table 1 in the paper’s layout. Compare the ordering and trend against the paper rather than the magnitudes. `plots/plot_all.py` reports which figures it skipped and why.

7 What this artifact does and does not reproduce

7.1 Reproduced

- **Figure 3**, interposition overhead.
- **Figure 5**, dynamic I/O provisioning, including the Overseer’s max-min fair share loop: modules register a demand, the Overseer computes `token_rate`, and each module’s token bucket applies it. Modules deregister on exit and the policy recomputes.
- **Figure 6**, in-runtime Parquet batching.
- **Figure 7**, hostcall batching.
- **Table 1**, the `s3-sync-ctx` and `s3-async-ctx` rows.

Details, including expected output, are under [Step-by-Step Instructions](#).

7.2 Not reproduced, and why

- **geds-ctx** (Table 1) needs a native IBM GEDS installation. Source is in `crates/contexts/peridot-geds-ctx/`; `docs/peridot.md` documents the build.
- **s3fs** (Table 1) needs FUSE and a privileged container. It is the row the paper normalises to, so `plots/plot_batch.py` normalises to the slowest arm present and says so.
- **Figure 4**, Overseer scalability, needs two nodes of a specific size, and the `UpdateMetrics` RPC carries no timestamp, so there is no latency instrumentation to read.

7.3 Magnitudes

Magnitudes differ from the paper, which ran on AWS EC2 against Amazon S3: `async-vs-sync` is 2.2x here against 5.1x there, and the `hostcall-batching` speedup reaches 514x against 4.90x because it is measured against a baseline that is slower inside a VM. Orderings and trends are unaffected.

8 Deviations from the paper’s method

1. **Figure 5, baseline arm.** Each baseline module gets a fixed, equal 250 Mbps bucket and no Overseer, reproducing the paper’s fixed aggregate cap and equal split without cgroups or a privileged container.
2. **Figure 6 and Table 1, timing.** Arms are timed by wall clock until the run’s data is durably in the object store. The guest’s own “Preprocessed images in N ms” line is not used: it stops before the asynchronous uploads do, and excludes the external batcher’s work entirely.
3. **s3-sync-ctx was added for this artifact.** Only the asynchronous S3 context was in the repository. It is now a `sync: true` option on the `s3` context, which awaits each upload inside `fd_write` instead of spawning it.
4. **The ImageNet guest takes a destination prefix** as an optional third argument; it previously hardcoded a bucket name.
5. **The IOPS guest is parameterised.** `c-wasm/write_1g` hardcodes 4,194,304 writes and 30 repetitions, hours per configuration. `c-wasm/iops` takes both as arguments.
6. **S3 path-style addressing was added.** The AWS SDK addresses buckets virtual-hosted-style, which a self-hosted store reached by hostname cannot serve. The `s3` and `batch` contexts and `tensor_batcher` honour `AWS_S3_FORCE_PATH_STYLE`, set by the compose file and off by default.

9 Layout

<code>artifact/</code>	
<code>Dockerfile</code>	multi-stage build, invoked by docker compose
<code>docker-compose.yml</code>	Peridot + MinIO
<code>env.example</code>	copy to <code>.env</code> to override credentials or endpoint
<code>guests/</code>	prebuilt <code>.wasm</code> modules
<code>configs/</code>	example Peridot configuration files
<code>experiments/</code>	one script per figure, each writing a CSV
<code>plots/</code>	one matplotlib script per figure
<code>results/</code>	CSVs, mounted from the host
<code>results/reference/</code>	sample data, for <code>--reference</code>
<code>figures/</code>	generated PDFs and PNGs, mounted from the host
<code>bin/</code>	guest and PDF build scripts

9.1 Guest modules

Module	Source	Used by
<code>hostcall_latency.wasm</code>	<code>c-wasm/hostcall_latency</code>	Figure 3
<code>io_writer.wasm</code>	<code>c-wasm/io_writer</code>	Figure 5, smoke test
<code>iops.wasm</code>	<code>c-wasm/iops</code>	Figure 7
<code>imagenet-preprocessing.wasm</code>	<code>crates/contexts/imagenet-preprocessing</code>	Table 1, Figure 6
<code>read_write.wasm</code> , <code>helloworld.wasm</code>	<code>c-wasm/</code>	ad-hoc use

Module	Source	Used by
--------	--------	---------

`bin/build-guests.sh` rebuilds them from source. It needs `wasi-sdk` and is not required to run the experiments.

10 Running without Docker

Linux only. The Overseer’s metrics producer reads `/proc/<pid>/io`, so on a host without `/proc` its thread dies on the first tick and each module keeps the rate from its own configuration file. The Figure 5 plot then does not demonstrate the Overseer. Other experiments are unaffected.

Needs the dependencies in `docs/peridot.md` (`protoc`, `cmake`, a C toolchain):

```
cargo build --release --locked -p peridot-cli -p peridot-overseer \
    -p tensor_batcher -p peridot-trace-plugin
pip install -r artifact/plots/requirements.txt boto3

cd artifact
experiments/smoke.sh
S3_ENDPOINT=http://127.0.0.1:9000 AWS_ACCESS_KEY_ID=... \
    AWS_SECRET_ACCESS_KEY=... experiments/run_all.sh
```

Build the runtime and any plugin **in one cargo invocation**, as above. Contexts cross the plugin boundary as ordinary Rust types with no stable ABI, and Cargo resolves features per invocation, so separate builds can disagree and the plugin will fail to load.

11 The containerd shim

`containerd-shim-peridot` runs a Wasm workload as an ordinary container under `ctr`, `nerdctl` or Kubernetes. It cannot run inside this image: it needs a privileged Linux host with `containerd` and a registry. `crates/containerd-shim-peridot/README.md` has the walkthrough.

12 Acknowledgements



This work was partially conducted during Pablo Gimeno Sarroca’s internship at IBM Research Zürich as part of the CLOUDSTARS EU mobility project (101086248). Supported by the European Union through the projects SIXG (101291424) and CloudSkin (101092646), and by the Spanish MICIU/AEI (PID2023-148202OB-C21). With the support of the Joan Oró predoctoral grant program from the Department of Research and Universities of the Government of Catalonia and co-financing by the European Social Fund Plus.